# Chapter 19 — Responsible ML: Fairness, Explainability, Privacy and Security

Every chapter so far asked "is the model accurate?". This one asks the questions that decide whether a model *should ship*: Is it fair to every group it touches? Can we explain its decisions? Does it leak the private data it learned from? Can someone fool it, poison it, or steal it? Which laws apply?

These are not side topics. A credit model that quietly rejects qualified applicants from one neighbourhood, or a medical model that reveals who was in its training set, is *broken* however good its AUC (the standard score of how well a model ranks positive cases above negative ones). Such failures have measurable symptoms and concrete fixes, and this chapter treats them as engineering problems.

**What you'll learn**

1. Where bias enters an ML system, from historical data to feedback loops.
2. The main fairness definitions, why they cannot all hold at once, and how to measure them in code.
3. Mitigation before, during and after training: reweighing, fairness penalties, group thresholds.
4. Explainability: permutation importance, partial dependence, SHAP (computed exactly) and LIME.
5. Privacy: why "anonymised" data often isn't, k-anonymity, differential privacy, federated learning.
6. Security: adversarial examples (FGSM), poisoning, model extraction and inversion, membership inference, generative-AI misuse.
7. Model cards, datasheets, regulation (GDPR — the EU's data-protection law — the EU AI Act, US rules), real incidents and a pre-launch checklist.

All code is in `code/` (one script per section), figures in `img/`. Every output shown is real (Python 3.14, scikit-learn 1.9, PyTorch 2.14, NumPy 2.5 — Python's standard classic-ML, deep-learning and array-maths libraries). The optional libraries `shap`, `lime` and `fairlearn` are not installed in the book environment, so their snippets are marked **(not run)**; each has a runnable from-scratch equivalent.

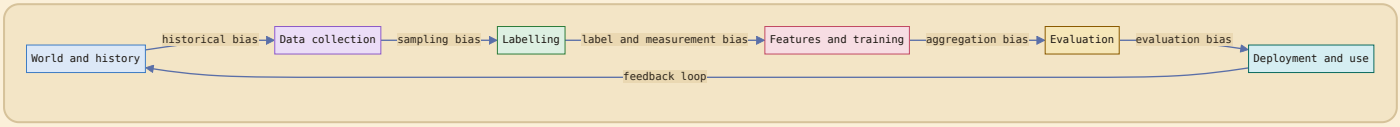



*Figure — bias can enter at every stage of the ML lifecycle, and deployment feeds back into the next round of data.*

---

## 19.1 Where bias comes from

### What it is

In this chapter **bias** means a *systematic* difference in how a model treats or performs for different groups of people that is not justified by the task. (This is different from the statistical "bias" in the bias–variance trade-off of Chapter 6.) A **protected** or **sensitive attribute** is a characteristic that anti-discrimination law or ethics says should not drive decisions — e.g. race, sex, age, religion, disability. A **group** is the set of people sharing a value of that attribute.

Analogy: a model is an apprentice who learns only by watching past decisions. If those decisions were unfair, or it only ever watched one kind of customer, it reproduces the unfairness — at scale and with an air of objectivity.

### Why it matters

Data records human choices and who was measured. Three mechanisms matter most:

1. **Data encodes past decisions.** Hiring, lending and arrest records carry the inequalities of their time: a résumé screener trained on "who we hired" learns "who we used to prefer", not "who does the job well".
2. **Some groups are thin in the data.** The model sees few of their examples, and the training loss (the error score that training tries to push down) barely notices errors on them. Classic result: the *Gender Shades* study (Buolamwini & Gebru, 2018) found commercial gender classifiers erred on up to 34.7% of darker-skinned women versus at most 0.8% of lighter-skinned men.
3. **Stereotyped associations get learned and amplified.** Word embeddings (vectors of numbers that represent words, learned from which words appear together) trained on web text place "nurse" near "she" and "programmer" near "he" (Bolukbasi et al., 2016). Anything built on top — job recommendations, ad targeting, autocomplete — inherits the association.

The harm (denied loans, misidentification, worse service) tends to land on people who were already disadvantaged; for the organisation it means legal exposure and lost trust.

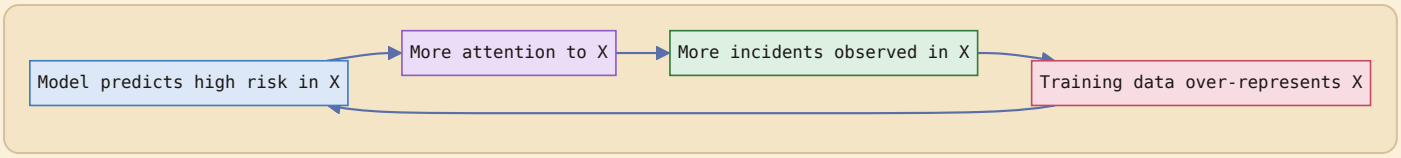
## How it works — a taxonomy of sources

(Names follow Suresh & Guttag's widely used framework.)

| Source | What goes wrong | Example |
|---|---|---|
| **Historical bias** | The world itself was unequal; even perfect data reproduces it | Past hiring favoured men, so "good hire" labels skew male |
| **Sampling / representation bias** | Some groups are under-sampled or missing | Face datasets dominated by lighter-skinned faces; a voice assistant trained mostly on one accent |
| **Measurement bias** | A feature or proxy (a stand-in for something you can't measure directly) is measured differently, or means different things, across groups | "Number of prior arrests" reflects policing intensity, not just behaviour |
| **Label bias** | The target is a flawed proxy, or is recorded unevenly | Health *cost* used as a stand-in for health *need* (Obermeyer et al., 2019): less was spent on Black patients at the same level of illness, so the algorithm under-rated their need |
| **Aggregation bias** | One model for groups whose feature–outcome relationship differs | A single HbA1c (a blood-sugar test) threshold across ethnicities with different typical values |
| **Evaluation bias** | The benchmark doesn't represent the users; only an overall metric is reported | 95% overall accuracy hiding 70% accuracy on a minority group |
| **Deployment bias** | The model is used for a purpose or population it was not built for | A risk score designed to target help being used to set punishment |
| **Feedback loops** | The model's decisions shape the next training data | Predictive policing (below) |

**Feedback loops.** Predictive policing is the textbook case (Lum & Isaac, 2016, simulated it on Oakland drug-crime data): the model flags district X, partly because X was patrolled more in the past; more officers go there, so more incidents are *recorded* there; the next training set shows even more crime in X. Nothing in the loop checks whether X really has more crime — the data only reflects where you looked. The same pattern appears in lending (people never approved never get a chance to show they

would repay — the **selective labels** problem) and in recommenders (items never shown never get clicks).




**Representative data is the first defence.** A model only works for people it has seen: coverage across ages, accents, skin tones, dialects, devices and regions gives it examples of every kind of real user and of rare cases.

### Common mistakes

- Assuming a model is neutral because the algorithm is "just maths" — the data carries the bias.
- Checking only overall accuracy; per-group performance is where bias shows.
- Assuming more data fixes it — more of the same skewed data makes the model *more confident* in the skew.
- Ignoring the label: if the target itself is biased (arrests, past hires, cost), no model choice rescues it.

### In the interview

**30-second answer:** "Bias usually comes from data, not the algorithm: past decisions, thin coverage of some groups, proxy labels or features, and feedback loops. I audit per-group coverage and metrics, question whether the label measures what we care about, and watch for loops after launch."

- *How do you break a feedback loop?* → Keep a small randomised exploration slice, log what you *didn't* show or approve, and monitor whether input distributions drift towards the model's own predictions.

### Self-check

1. A hiring model is trained on which applicants managers promoted. Which bias is most likely? → Historical and label bias: "promoted" encodes past managers' preferences.
2. Your speech model works well overall but poorly for Scottish accents. Which bias? → Sampling/representation bias (and evaluation bias if nobody measured per-accent accuracy).
3. Why can't you fix a feedback loop just by retraining more often? → Retraining on the loop's own output reinforces it; you need exploration or external ground truth (the real, independently verified outcome).

### TL;DR

Bias enters at every stage, from history to feedback loops. Representative data and per-group evaluation come first; no algorithm fixes a biased label.

---

## 19.2 Fairness definitions and the impossibility trade-off

### What it is

A **fairness criterion** is a precise statistical condition on a model's predictions across groups. Notation used below: **A** is the group, **Y** the true outcome (1 = good, e.g. repays), **Ŷ** the decision (1 = approve), **S** the model's score. TPR (true-positive rate, recall), FPR (false-positive rate), FNR = 1 − TPR and PPV (precision) are the confusion-matrix rates (built from the counts of true/false

positives and negatives) from Chapter 6, computed separately per group. Most criteria fall into three families (Barocas, Hardt & Narayanan's *Fairness and Machine Learning*):

- **Independence** — the decision doesn't depend on the group.
- **Separation** — errors don't depend on the group, given the true outcome.
- **Sufficiency** — a given score means the same thing in every group.

| Criterion | Family | Condition (equal across groups) | Plain meaning |
|---|---|---|---|
| **Demographic (statistical) parity** | Independence | P(Ŷ=1 \| A=a) | Same approval rate in every group |
| **Disparate impact ratio** | Independence | P(Ŷ=1 \| B) / P(Ŷ=1 \| A) ≥ 0.8 | The US "four-fifths rule" of thumb from the 1978 Uniform Guidelines on employee selection |
| **Equal opportunity** | Separation (half) | TPR = P(Ŷ=1 \| Y=1, A=a) | Qualified people are approved at the same rate |
| **Equalized odds** | Separation | TPR *and* FPR = P(Ŷ=1 \| Y=0, A=a) | Same hit rate and same false-alarm rate |
| **Predictive parity** | Sufficiency (at the threshold) | PPV = P(Y=1 \| Ŷ=1, A=a) | An approval is equally reliable in each group |
| **Calibration within groups** | Sufficiency | P(Y=1 \| S=s, A=a) = s for all s | A score of 0.7 means a 70% chance, whatever the group |

Two other ideas you should know by name:

- **Individual fairness** (Dwork et al., 2012): similar individuals should get similar predictions. Hard in practice because you must define "similar".
- **Counterfactual fairness** (Kusner et al., 2017): the prediction should not change if the person's group were different, with everything causally downstream of it adjusted. Needs a causal model of the data (a map of which variables cause which).

### Why it matters

"Make it fair" is not actionable; "equalise TPR between groups within 2 points" is. Picking a definition is a **values decision** tied to the harm you care about:

- Hiring shortlist where the base rates (the true share of positives in each group) are themselves the product of discrimination → **demographic parity** may be justified.
- Loans, medical screening, where missing a qualified person is the key harm → **equal opportunity**.
- Criminal justice, where both false accusations and missed risks matter → **equalized odds**.
- Risk scores read by humans (a doctor, a judge) → **calibration**, so a number means the same thing for everyone.

### How it works — the impossibility result

When groups have **different base rates** (different true rates of Y=1), you generally cannot have calibration *and* equal FPR *and* equal FNR at the same time, unless the model is perfect. This was shown independently by Kleinberg, Mullainathan & Raghavan (2016) and Chouldechova (2017), prompted by the COMPAS debate (§19.10).

Chouldechova's identity makes it concrete. For any group with base rate p:

FPR = p / (1 − p) · (1 − PPV) / PPV · (1 − FNR)

If PPV and FNR are equal across two groups but p differs, the right-hand side differs, so FPR *must* differ.

`code/impossibility.py` builds scores that are **perfectly calibrated by construction** (each person's outcome is drawn with probability equal to their score) for two groups whose base rates are 0.6 and 0.4:

```
groups = {"A": rng.beta(6, 4, n), "B": rng.beta(4, 6, n)}   # mean score 0.6 vs 0.4
data = {g: (s, (rng.random(n) < s).astype(int)) for g, s in groups.items()}
# ... one shared threshold 0.5, then group thresholds chosen so FPR = 0.20 in both
```

Output:

```
group  base_rate  calib(0.7-0.8 bin)  FPR    FNR    PPV   selection
  A      0.601      0.746->0.748     0.624  0.171  0.667  0.747
  B      0.399      0.737->0.728     0.171  0.623  0.594  0.253

Group thresholds chosen so FPR = 0.20 in both groups:
  A: threshold=0.673  FPR=0.200  FNR=0.574  PPV=0.762
  B: threshold=0.483  FPR=0.200  FNR=0.582  PPV=0.582
```

Reading it: both groups are calibrated (scores near 0.74 come true about 74% of the time), yet one shared threshold gives FPR 0.62 for A and 0.17 for B. Group thresholds equalise FPR (and here FNR too), but then PPV splits, 0.76 vs 0.58: an approval "means" different things per group. Check the identity for group A at threshold 0.5: 0.601/0.399 × 0.333/0.667 × 0.829 ≈ 1.506 × 0.499 × 0.829 ≈ **0.623** — matches the measured FPR of 0.624.

## Trade-offs

| If you enforce… | You typically give up… |
|---|---|
| Demographic parity | Calibration and accuracy when base rates truly differ; may approve less-qualified people in one group |
| Equalized odds | Calibration / predictive parity; usually needs group-specific thresholds |
| Calibration | Equal error rates when base rates differ |
| Any constraint | Some overall accuracy (often small, see §19.4) |

## When not to use a given metric

- Demographic parity, when base-rate differences are real and legitimate (disease prevalence by age).
- Equal opportunity, when the labels themselves are biased — you would equalise against a distorted target.
- The 4/5 rule as a legal safe harbour (a rule that guarantees you're legally in the clear); it is only a screening heuristic.

## Common mistakes

- Reporting one fairness number without saying which definition.
- Comparing groups with tiny sample sizes — gaps of 10 points can be noise; report confidence intervals (a range likely to contain the true value).
- Forgetting **intersectional** groups (e.g. older women), where gaps are often largest.

## In the interview

**30-second answer:** "Criteria fall into independence (demographic parity), separation (equal opportunity, equalized odds) and sufficiency (predictive parity, calibration). With different

base rates, calibration and equal error rates can't both hold (Kleinberg; Chouldechova), so I pick the criterion that matches the harm, document why, and report the rest."

- *Which would you pick for loan approvals?* → Usually equal opportunity (qualified applicants approved at equal rates), plus calibration checks on the score; confirm with legal and domain experts.
- *Does the impossibility mean fairness is hopeless?* → No; trade-offs must be chosen explicitly, and a more accurate model shrinks all the gaps.

### Self-check

1. Group A approval 50%, group B 35%. Disparate impact ratio? → 0.35/0.50 = 0.70, below 0.8.
2. Equalized odds requires equality of which two rates? → TPR and FPR.
3. Why can't a calibrated model have equal FPRs when base rates differ? → By FPR = $p/(1-p)\cdot(1-\text{PPV})/\text{PPV}\cdot(1-\text{FNR})$, different p forces a different FPR, PPV or FNR.

### TL;DR

Parity = equal approval rates; equal opportunity = equal TPR; equalized odds = equal TPR and FPR; calibration = scores mean the same everywhere. With unequal base rates they conflict: choose by harm and report the rest.

---

## 19.3 Measuring fairness in code

### What it is

A **fairness audit** computes the confusion-matrix rates separately per group and compares them. It needs the true label, the model's decision (and score) and the group, even if the model never sees the group.

### The problem it solves

Overall metrics hide group gaps; an audit makes them visible and trackable, like accuracy.

### How it works

1. Hold out a test set that includes the sensitive attribute (collected with consent for auditing).
2. Per group compute n, base rate, selection rate, TPR, FPR, PPV and accuracy.
3. Compute the gaps: demographic-parity difference, disparate-impact ratio, equal-opportunity difference, equalized-odds difference (the larger of the TPR and FPR gaps), predictive-parity difference.
4. Check calibration per group, and check for **proxies**: how well do the other features predict the group?

**The dataset.** `code/credit_data.py` generates 20,000 synthetic loan applicants whose true repayment chance depends *only on finances*. Two biases are injected: group B has lower average income (historical inequality), and in the *recorded* labels 20% of B applicants who actually repaid are marked as defaults (label bias). `zip_region` correlates strongly with group — a proxy.

```
       income_k  zip_region  y_true      y
group
0        58.048       0.145   0.534  0.534
1        45.310       0.796   0.425  0.340
```

The core of `code/fairness_metrics.py`:

```
def group_report(y, y_hat, group):
    rows = []
    for g in np.unique(group):
        m = group == g
        yt, yp = y[m], y_hat[m]
        tp = np.sum((yt == 1) & (yp == 1)); fp = np.sum((yt == 0) & (yp == 1))
        fn = np.sum((yt == 1) & (yp == 0)); tn = np.sum((yt == 0) & (yp == 0))
        rows.append({"group": "AB"[g], "n": m.sum(), "base_rate": yt.mean(),
                     "selection_rate": yp.mean(), "TPR": tp / (tp + fn),
                     "FPR": fp / (fp + tn), "PPV": tp / (tp + fp),
                     "accuracy": (tp + tn) / m.sum()})
    return pd.DataFrame(rows).set_index("group")

def fairness_summary(r):
    a, b = r.loc["A"], r.loc["B"]
    return {"demographic_parity_diff": a.selection_rate - b.selection_rate,
            "disparate_impact_ratio": b.selection_rate / a.selection_rate,
            "equal_opportunity_diff": a.TPR - b.TPR,
            "equalized_odds_diff": max(abs(a.TPR - b.TPR), abs(a.FPR - b.FPR)),
            "predictive_parity_diff": a.PPV - b.PPV}

# the model is "group-blind": FEATURES excludes `group`
model = make_pipeline(StandardScaler(), LogisticRegression()).fit(train[FEATURES], train.y)
```

Output:

```
Group report vs RECORDED labels (what an auditor can see):
          n  base_rate  selection_rate    TPR    FPR    PPV  accuracy
group
A      4245      0.538           0.516  0.709  0.292  0.739     0.709
B      1755      0.349           0.281  0.436  0.198  0.542     0.675
  demographic_parity_diff  +0.235
  disparate_impact_ratio   +0.544
  equal_opportunity_diff   +0.273
  equalized_odds_diff      +0.273
  predictive_parity_diff   +0.197

Same predictions vs the hidden TRUE labels:
       base_rate  selection_rate    TPR    FPR
group
A          0.538           0.516  0.709  0.292
B          0.435           0.281  0.439  0.159

AUC for predicting group from the 'group-blind' features: 0.886
calibration A: (0.13->0.13) (0.30->0.33) (0.50->0.53) (0.70->0.73) (0.89->0.89)
calibration B: (0.12->0.14) (0.30->0.30) (0.50->0.45) (0.69->0.59) (0.87->0.66)
```

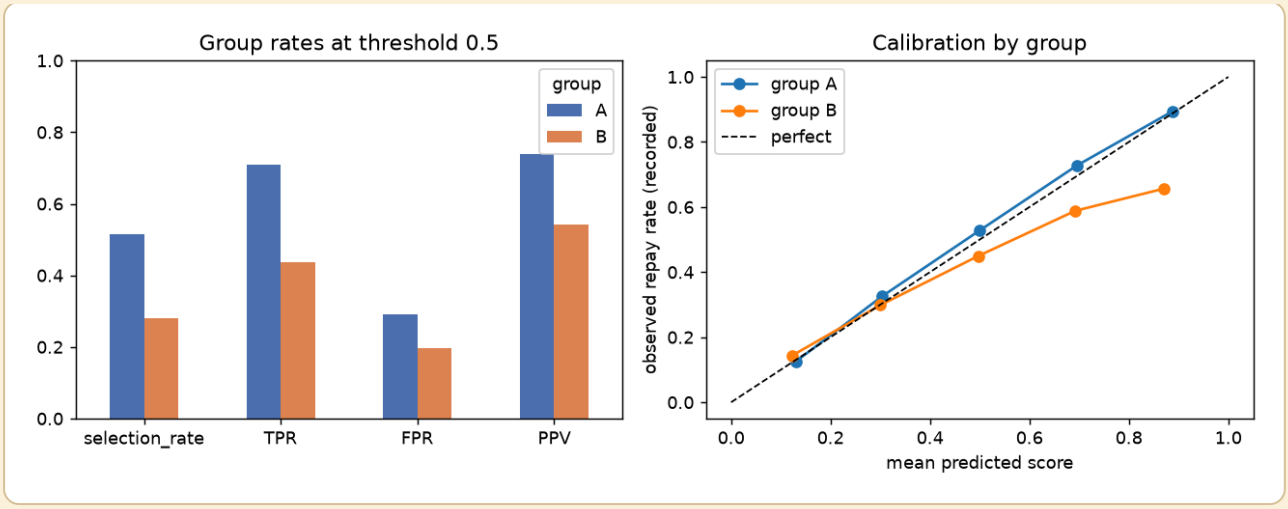



*Figure — left: selection rate, TPR, FPR and PPV per group at threshold 0.5. Right: calibration curves; group B's curve sags below the diagonal at high scores.*

What the numbers say:

1. **Every criterion fails.** Group B is approved 28% of the time vs 52% (ratio 0.54, far below 0.8); qualified B applicants are approved 44% of the time vs 71%.
2. **"Group-blind" didn't work.** The model never saw `group`, yet the other features predict it with AUC 0.886, mostly via `zip_region`. Dropping the sensitive column (**fairness through unawareness**) does not remove its influence and stops you measuring it.
3. **Label bias is invisible to the auditor.** On recorded labels B's base rate looks like 0.349; the truth is 0.435. Only knowing *how labels were produced* reveals that some B "defaults" are artefacts.
4. **Calibration differs.** A score of 0.89 comes true 89% of the time for A but only 66% for B (recorded labels), so a human reading the score is misled for one group.

The same audit with Fairlearn's `MetricFrame` **(not run — `fairlearn` not installed)**:

```
from fairlearn.metrics import MetricFrame, selection_rate, true_positive_rate, false_positive_rate
mf = MetricFrame(metrics={"sel": selection_rate, "tpr": true_positive_rate, "fpr":
false_positive_rate},
                 y_true=y_test, y_pred=y_pred, sensitive_features=group_test)
print(mf.by_group); print(mf.difference())
```

## Trade-offs

| Choice | Pro | Con |
|---|---|---|
| Collect the sensitive attribute | Lets you measure and fix bias | Privacy risk; some jurisdictions restrict it; needs consent and access controls |
| Don't collect it | Simpler legally | Bias still happens through proxies — you just can't see it |
| Infer it (e.g. BISG — Bayesian Improved Surname Geocoding — surname + geography method used by US regulators) | Enables audits without collection | Error-prone; must be used only for auditing |

## Common mistakes

- Auditing on the training set instead of held-out data (examples kept aside and never used for training).
- Reporting gaps without group sizes and confidence intervals.

- Treating recorded labels as ground truth when the labelling process itself is suspect.
- Checking fairness once before launch and never again — drift (the live population or data shifting after launch) changes gaps.

### Real-world examples

Fairlearn (started at Microsoft) and IBM's AIF360 are open-source fairness toolkits; Google's What-If Tool visualises per-group metrics. US lenders run fair-lending analyses on underwriting (deciding whether to lend, and on what terms) models for regulatory compliance.

### In the interview

> **30-second answer:** "I slice held-out data by group, compute selection rate, TPR, FPR, PPV and calibration per group and the gaps between groups, and predict the sensitive attribute from the features to find proxies. I report group sizes and confidence intervals and keep tracking the metrics in production."

- *Why keep the sensitive attribute if the model doesn't use it?* → To measure and mitigate; you can store it separately with strict access controls.
- *How do you find proxies?* → Predict the group from features (AUC), check feature–group correlations, and inspect feature importances.

### Self-check

1. Equalized-odds difference with TPR gap 0.27 and FPR gap 0.09? → 0.27 (the maximum).
2. What does an AUC of 0.886 for predicting group from features mean? → The features encode group membership strongly — proxies exist.
3. Why is dropping the sensitive column called "fairness through unawareness", and why is it weak? → The model is unaware of the column, but proxies carry the same information.

### TL;DR

Slice metrics by group, compute the gaps and per-group calibration, test for proxies, and never trust "we removed the sensitive column".

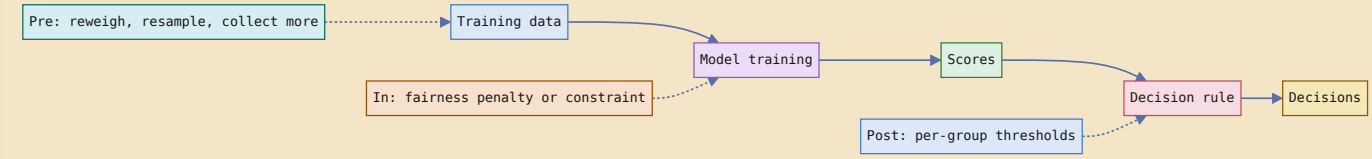
---

## 19.4 Mitigating bias: before, during and after training

### What it is

**Bias mitigation** changes the data, the training objective or the decision rule so that a chosen fairness criterion improves, at an acceptable cost in accuracy. The three intervention points:

| Stage | Idea | Techniques |
|---|---|---|
| **Pre-processing** (fix the data) | Change what the model learns from | Collect more data for thin groups, re-sample, **reweighing** (Kamiran & Calders, 2012), relabel suspect labels, learn fair representations, synthetic augmentation (adding generated examples) for rare groups |
| **In-processing** (fix the training) | Add fairness to the objective | **Fairness penalties / constraints** (e.g. Zafar et al., 2017), reductions approach (Agarwal et al., 2018; Fairlearn's `ExponentiatedGradient`; turns the constraint into a series of reweighted ordinary training runs), adversarial debiasing (Zhang et al., 2018; a second network tries to guess the group from the model's output and the model learns to defeat it) |
| **Post-processing** (fix the decisions) | Keep the model, change how scores become decisions | **Group-specific thresholds** (Hardt, Price & Srebro, 2016), reject-option classification (flip uncertain decisions near the threshold in favour of the disadvantaged group) |




### Why it matters

Measurement without mitigation only documents harm. Post-processing needs no retraining (good for a vendor model); pre-processing leaves training code untouched; in-processing usually gives the best accuracy–fairness trade-off but needs custom training.

## How it works

**Reweighing.** Give each (group, label) cell a weight so that, in the weighted data, label and group are statistically independent (knowing one tells you nothing about the other):

$$w(g, y) = P(A=g) \cdot P(Y=y) / P(A=g, Y=y)$$

Cells that are rarer than independence predicts (here: group B with a good outcome) get weights above 1. The model is then trained with `sample_weight=w`.

**Fairness penalty.** Add a term to the loss that grows with the gap you want to close. Here: logistic loss + $\lambda \cdot$(mean score of A − mean score of B)$^2$. Larger $\lambda$ means more parity, less accuracy.

**Threshold adjustment (post-processing).** Keep the scores; choose a separate threshold per group, fitted on a *validation* set (held-out data used for tuning choices, separate from the final test set), so the chosen rate matches:

1. Fix group A's threshold (0.5).
2. Measure the target rate for A on validation data — its TPR (for equal opportunity) or selection rate (for demographic parity).
3. For group B pick the threshold at the matching quantile (the score below which a given fraction of values falls) of its scores (among its true positives for TPR, among all its members for selection rate).
4. Apply both thresholds on the test set and re-audit.

Key code from `code/mitigation.py`:

```python
def reweighing_weights(y, g):
    w = np.empty(len(y))
    for gv in (0, 1):
        for yv in (0, 1):
            m = (g == gv) & (y == yv)
            w[m] = (np.mean(g == gv) * np.mean(y == yv)) / np.mean(m)
    return w

def fair_logreg(X, y, g, lam, epochs=300, seed=0):
    # ... torch Linear layer + Adam
        p = torch.sigmoid(logits)
        gap = p[gA].mean() - p[gB].mean()
        loss = F.binary_cross_entropy_with_logits(logits, y_t) + lam * gap ** 2

def threshold_for_rate(score, target, positives_only_mask=None):
    s = score if positives_only_mask is None else score[positives_only_mask]
    return np.quantile(s, 1 - target)           # the top `target` fraction clears it

# equal opportunity: B's threshold makes TPR_B (validation) = TPR_A
tpr_A = np.mean(s_va[(gva == 0) & (yva == 1)] >= 0.5)
tB = threshold_for_rate(s_va[gva == 1], tpr_A, yva[gva == 1] == 1)
y_eo = np.where(gte == 0, s_te >= 0.5, s_te >= tB).astype(int)
```

Output (`acc` against recorded labels, `acc_true` against the hidden true labels; `EO_diff` = TPR gap):

```
reweighing weights  (A,y=0) (A,y=1) (B,y=0) (B,y=1): [1.123, 0.892, 0.791, 1.413]
equal-opportunity thresholds: A=0.500  B=0.356
demographic-parity thresholds: A=0.500  B=0.340
                           acc  acc_true  sel_A  sel_B  DP_diff  EO_diff  EO_diff_true
method
baseline (t=0.5)         0.704     0.701  0.532  0.269    0.263    0.290         0.291
pre: reweighing          0.703     0.704  0.501  0.349    0.153    0.155         0.159
in: DP penalty (lam=5)   0.690     0.695  0.472  0.420    0.053    0.055         0.059
post: equal opportunity  0.694     0.702  0.532  0.481    0.051    0.041         0.044
post: demographic parity 0.690     0.699  0.532  0.511    0.021    0.017         0.024
```

Reading it:

1. **Reweighing** roughly halves both gaps with no accuracy loss (group-B positives get weight 1.41).
2. **The penalty** closes the parity gap to 5 points for about 1.4 points of recorded accuracy.
3. **Threshold adjustment** closes the TPR gap from 29 to 4 points simply by lowering B's threshold to 0.356; accuracy on the *true* labels is unchanged (0.701 → 0.702), because part of the "unfairness" was the label bias.
4. The accuracy cost of fairness is often **smaller than expected**, but it is data-dependent: measure it.

Fairlearn equivalents **(not run — `fairlearn` not installed)**:

```python
from fairlearn.postprocessing import ThresholdOptimizer
from fairlearn.reductions import ExponentiatedGradient, EqualizedOdds
post = ThresholdOptimizer(estimator=base_model, constraints="true_positive_rate_parity",
                          prefit=True, predict_method="predict_proba")
post.fit(X_val, y_val, sensitive_features=g_val)
y_hat = post.predict(X_test, sensitive_features=g_test, random_state=0)

inproc = ExponentiatedGradient(LogisticRegression(), constraints=EqualizedOdds())
inproc.fit(X_train, y_train, sensitive_features=g_train)
```

**Process controls matter as much as algorithms:** audit who is in the data (per group and intersection) before training; collect more examples for thin groups, using synthetic data only as a supplement; give humans the authority to review and override high-stakes decisions; and document data choices so outsiders can audit them.

## Trade-offs

| Method | Pros | Cons |
|---|---|---|
| Reweighing / resampling | Simple, model-agnostic, no group needed at prediction time | Limited control over which criterion improves |
| Fairness penalty / constraints | Directly targets a criterion; good trade-offs | Custom training; hyperparameter (a setting you choose rather than learn) λ; can be unstable |
| Group thresholds | No retraining; precise control; works on vendor models | Uses the group at decision time — **may be unlawful disparate treatment** (explicitly treating people differently because of their group) in some settings (e.g. US employment law) |
| Collecting better data / labels | Fixes the cause | Slow and costly |

## When not to use it

Don't bolt on post-processing when the real issue is a broken label (fix the label), and don't deploy group thresholds without legal review.

## Common mistakes

- Fitting thresholds on the test set (leaks information; always use a validation set).
- Tiny groups: thresholds become noisy — pool data or use Bayesian estimates (which blend a small group's data with what the whole population suggests).
- Declaring victory on one metric while another gets worse.

## Real-world examples

LinkedIn has described re-ranking for gender-representative candidate search results in LinkedIn Recruiter. Fairlearn's `ThresholdOptimizer` implements Hardt et al.'s post-processing; AIF360 implements reweighing, adversarial debiasing and reject-option classification.

## In the interview

> **30-second answer:** "Pre-processing reweighs or improves the data; in-processing adds a fairness penalty or constraint to the loss; post-processing sets per-group thresholds on validation data. Post-processing is cheap and precise but uses the attribute at decision time, which has legal implications. Afterwards I re-audit every metric and the accuracy cost."

- *Which is best?* → Fix data and labels first; in-processing usually gives the best trade-off; post-processing when you can't retrain.
- *How big is the accuracy cost?* → Measure it; often a point or two, as here, especially when labels are biased.

## Self-check

1. P(B)=0.3, P(Y=1)=0.5, P(B, Y=1)=0.1. Reweighing weight for (B, 1)? → 0.3·0.5/0.1 = 1.5.

2. Why fit thresholds on validation, not test? → To avoid overfitting (tuning so closely to one dataset that it fails on new data) the fairness adjustment to the evaluation data.
3. Which stage suits a model bought from a vendor? → Post-processing.

### TL;DR

Pre-processing fixes data (reweighing), in-processing fixes the loss (fairness penalty), post-processing fixes the decision (group thresholds). Measure the accuracy cost and re-audit every criterion.

---

## 19.5 Explainability

### What it is

**Explainability** (or **interpretability**) is the ability to say *why* a model made a prediction in terms a human can check. A **global** explanation describes the model's overall behaviour (which features matter, and how); a **local** one explains a single prediction (why *this* applicant was declined). An **intrinsically interpretable** model is readable directly (linear models, small trees, rule lists, GAMs, i.e. generalised additive models); a **post-hoc** method explains a black box (a model whose inner workings a person can't read) from outside.

Analogy: a global explanation is a restaurant's recipe book; a local one is why tonight's dish tasted salty.

#### Why it matters

- **Debugging**: spot leakage (training inputs that secretly contain the answer), shortcuts (spurious cues the model latched onto) and proxies.
- **Trust**: doctors, underwriters and judges need reasons.
- **Law**: US lenders must state the principal reasons for denying credit (ECOA/Regulation B — the US Equal Credit Opportunity Act and its implementing rule — adverse-action notices); GDPR requires meaningful information about the logic of solely automated significant decisions (§19.9).
- **Recourse**: telling a person what would change the decision.

### How it works

**1. Permutation importance (global).**

1. Score the fitted model on held-out data (e.g. ROC-AUC).
2. Shuffle one column, breaking its link with the target; score again.
3. Importance = drop in score; repeat several shuffles and average.

Unlike tree impurity importance (Chapter 7; how much a feature's splits purify the tree's nodes), it is model-agnostic (works with any model) and measured on held-out data.

**2. Partial dependence (PDP) and ICE (global/local).** $PD(x_s) = (1/n)\ \Sigma_i\ f(x_s, x_c^{(i)})$: set feature s to a value for *every* row, average the predictions, repeat across a grid. **ICE** (individual conditional expectation) curves plot the same for each row separately; the PDP is their average. ICE reveals interactions that the average hides.

**3. SHAP (local, adds up to global). Shapley values** come from cooperative game theory (Shapley, 1953): the features are "players" sharing the "payout" f(x) − E[f]. Feature i's value is its average marginal contribution (how much the prediction changes when that feature is added) over all orders in which features could be added:

$\varphi_i = \Sigma$ over subsets S not containing i of $|S|!(M-|S|-1)!/M! \cdot [v(S \cup \{i\}) - v(S)]$

where v(S) is the expected prediction when only the features in S are known (the others drawn from background data). The weight |S|!(M−|S|−1)!/M! is just the fraction of all M! orderings of the features in which exactly the features of S come before i, so $\varphi_i$ is i's average extra contribution over every order of arrival. Properties: **efficiency** (φ's sum to f(x) − E[f]), **symmetry**, **dummy** (a feature that never changes the output gets 0), **additivity**. Exact computation needs 2^M subsets, so the SHAP library (Lundberg & Lee, 2017) uses approximations: KernelSHAP (any model, sampling) and TreeSHAP (exact and fast for tree ensembles — models built from many decision trees).

**4. LIME (local).** Ribeiro et al., 2016: perturb (make small random changes to) the input around x, weight the black box's predictions by closeness to x, and fit a sparse linear model (one with few non-zero coefficients) whose coefficients are the explanation. Fast, but results shift with the kernel width (how fast the closeness weight falls off) and seed (the starting value of the random-number generator).

**5. Counterfactual explanations (local).** "You would have been approved if your debt ratio were 0.25 instead of 0.40" — the smallest *actionable* change that flips the decision.

`code/explainability.py` trains a histogram gradient-boosting model (boosted decision trees that bin feature values into buckets for speed) on the credit data (without `group`):

```
pi = permutation_importance(model, Xte, yte, scoring="roc_auc", n_repeats=10, random_state=0)
PartialDependenceDisplay.from_estimator(model, Xte, [0, 1, 4], feature_names=FEATURES,
                                        kind="both", subsample=300, random_state=0)

def exact_shapley(f, x, background):
    M = len(x)
    def v(S):                                   # expected output with features S fixed to x
        Z = background.copy(); Z[:, list(S)] = x[list(S)]
        return f(Z).mean()
    phi = np.zeros(M)
    for i in range(M):
        others = [j for j in range(M) if j != i]
        for size in range(M):
            for S in combinations(others, size):
                w = factorial(size) * factorial(M - size - 1) / factorial(M)
                phi[i] += w * (v(S + (i,)) - v(S))
    return phi, v(())

def lime_like(f, x, X_ref, n=5000, width=0.75, seed=0):
    # ... Gaussian samples around x, proximity weights, weighted Ridge on standardised offsets
```

Output:

```
test accuracy: 0.705

Permutation importance (drop in test ROC-AUC when a column is shuffled):
  income_k        0.1472 +/- 0.0044
  debt_ratio      0.0596 +/- 0.0028
  prior_defaults  0.0573 +/- 0.0035
  years_employed  0.0132 +/- 0.0019
  zip_region      0.0019 +/- 0.0010

Applicant: {'income_k': 70.9, 'debt_ratio': 0.27, 'years_employed': 10.0, 'prior_defaults': 1.0,
'zip_region': 0.0}
model score f(x) = 0.816   baseline E[f] = 0.471
Shapley values (push towards approval if +):
  income_k        +0.178
  years_employed  +0.120
  debt_ratio      +0.111
  prior_defaults  -0.090
  zip_region      +0.027
check: baseline + sum(phi) = 0.816  (equals f(x))

LIME-style local slopes (score change per +1 sd):
  income_k        +0.195
  debt_ratio      -0.089
  prior_defaults  -0.069
  years_employed  +0.055
  zip_region      -0.016
```

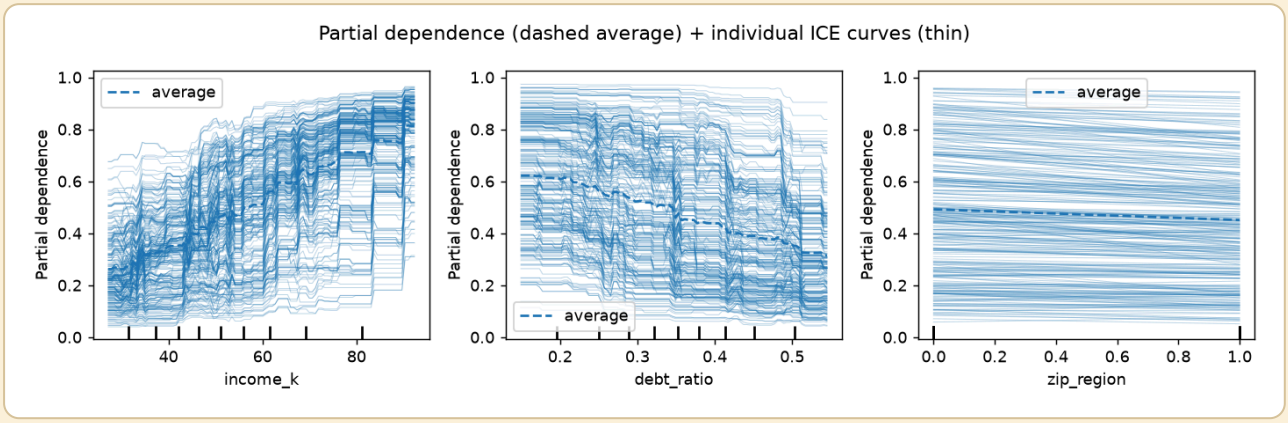



*Figure — PDP (dashed) and ICE curves (thin): approval probability rises with income, falls with debt ratio, and dips slightly for zip region 1.*

Reading it:

1. Income dominates globally. `zip_region` has small permutation importance — yet §19.3 showed it is a strong group proxy. **Low importance does not mean "fair"**; much of the group signal also flows through income.
2. For this applicant the Shapley values add up exactly: 0.471 + 0.178 + 0.120 + 0.111 − 0.090 + 0.027 = 0.816 (efficiency).
3. SHAP and LIME frame things differently but consistently: Shapley says "compared with an average applicant, living in zip 0 helped (+0.027)"; LIME's slope says "moving *towards* zip 1 would lower the score". (Our Gaussian perturbation of a 0/1 column is crude; the real LIME library samples categorical features properly.)

The SHAP library version **(not run — `shap` not installed)**:

```python
import shap
explainer = shap.TreeExplainer(model)          # exact, fast for tree ensembles
sv = explainer(X_test)                         # Explanation object, one row per sample
shap.plots.waterfall(sv[3])                    # local: one applicant
shap.plots.beeswarm(sv)                        # global: distribution of impacts per feature
```

## Trade-offs

| Method | Scope | Pros | Cons |
|---|---|---|---|
| Interpretable model | Both | Faithful by construction | May lose accuracy on complex data |
| Permutation importance | Global | Model-agnostic, uses held-out data | Misleading with correlated features (shuffling makes unrealistic rows; importance splits between twins) |
| PDP / ICE | Global / local | Shows shape of effect | Assumes feature independence; averages hide interactions (use ICE) |
| SHAP | Local → global | Principled, additive, consistent | Costly for non-tree models; depends on background data; still correlational |
| LIME | Local | Fast, any model | Unstable; depends on kernel and sampling |
| Counterfactuals | Local | Actionable recourse | Many possible answers; must respect feasibility |

## When not to use it

- Don't present post-hoc explanations as *causal* — they explain the model, not the world.
- For high-stakes decisions consider an interpretable model first (Rudin, 2019, argues this strongly).
- Don't show raw SHAP plots to customers; translate into plain-language reasons.

## Common mistakes

- Computing importance on training data (rewards overfitting).
- Ignoring correlated features — two twin features can each look unimportant.
- Treating one LIME run as the truth; check stability across seeds.
- Explaining the wrong output (log-odds — the logarithm of p/(1−p), a logistic model's raw output — vs probability) and confusing readers.

## Real-world examples

Lenders produce "reason codes" for adverse-action notices; the major cloud ML platforms (Vertex AI, SageMaker Clarify, Azure ML) offer SHAP-style attributions; TreeSHAP is built into XGBoost and LightGBM (popular gradient-boosting libraries) (`pred_contribs=True` / `pred_contrib=True`).

## In the interview

**30-second answer:** "Global explanations — permutation importance on held-out data, PDP and ICE — describe the whole model. Local ones explain a single prediction: SHAP splits it into additive feature contributions with game-theory guarantees; LIME fits a local linear surrogate (a simple stand-in model). Both are correlational, so I use them for debugging and reason codes, not causal claims."

- *SHAP vs LIME?* → SHAP has axioms (efficiency, symmetry, dummy) and consistent results; LIME is faster but less stable. TreeSHAP is exact for trees.

- *How do you explain a decline to a customer?* → Top negative SHAP contributors mapped to plain-language reason codes, plus a counterfactual if possible.

### Self-check

1. Baseline 0.40, SHAP values +0.10, −0.05, +0.20. Prediction? → 0.65.
2. Global or local: a PDP? a waterfall plot? → PDP global; waterfall local.
3. Why prefer held-out data for permutation importance? → Training-set importance rewards features the model overfit.

### TL;DR

Global: permutation importance, PDP. Local: SHAP (adds up to f(x) − E[f]), LIME, counterfactuals. Explanations describe the model, not causality.

---

## 19.6 Privacy

### What it is

**Privacy** in ML means controlling what personal information is collected, how it is used, and what the trained model can leak. **PII** (personally identifiable information) identifies a person directly (name, email, national ID) or in combination; **quasi-identifiers** such as ZIP code, birth date and sex are harmless alone but identifying together; **sensitive attributes** (health, finances, religion, biometrics) do great harm if revealed.

#### Why it matters

ML runs on data about people — photos, messages, purchases, scraped pages. Every team must answer:

1. **Consent and ownership.** Did people agree to their data training a model? If you send company data to a third-party AI service, is it used to train the vendor's models, and where can it flow next?
2. **Legal duties.** GDPR, California's CCPA/CPRA (its consumer-privacy laws) and similar laws require transparency, a lawful basis, and rights to access, delete or opt out.
3. **Harms.** Leaked data enables identity theft, harassment and discrimination; companies face fines and lost trust.
4. **Models memorise.** A trained model can reveal training records (§19.7), so "we only ship the model" is no privacy guarantee.

### How it works

**Anonymisation pitfalls.** Removing names is *pseudonymisation*, not anonymisation; GDPR still treats pseudonymised data as personal data. Famous failures:

- Latanya Sweeney estimated that 87% of Americans are likely unique on 5-digit ZIP + birth date + sex (1990 census), and re-identified the Massachusetts governor's hospital record by linking "anonymised" health data with a public voter roll.
- AOL's "anonymised" 2006 search logs let journalists identify individual users; Netflix Prize ratings were re-identified by matching them with public IMDb reviews (Narayanan & Shmatikov, 2008).
- Strava's 2018 global heatmap exposed military bases from soldiers' runs: aggregates leak too.

**k-anonymity** (Sweeney, 2002): every combination of quasi-identifiers must be shared by at least k records, achieved by **generalisation** (ZIP 02138 → 021**, age 29 → "≤35") and **suppression** (dropping outliers). Its weaknesses led to **l-diversity** (each group must contain at least l distinct sensitive values) and **t-closeness** (each group's sensitive-value distribution must be close to the overall one).

**Differential privacy (DP)** (Dwork et al., 2006) is a guarantee about an *algorithm*: its output distribution barely changes whether or not any one person's record is in the data. Formally, M is ε-DP if for all datasets D, D' differing in one record and every set of outputs O:

P[M(D) ∈ O] ≤ e^ε · P[M(D') ∈ O]

Small ε = strong privacy (ε = 1 means any outcome is at most e ≈ 2.72 times likelier with you in the data than without).

The **Laplace mechanism** achieves ε-DP for a numeric query f:

1. Compute the **sensitivity** Δf = the most one person can change f. A count has Δf = 1; a mean of values clipped to [L, U] over n people has Δf = (U − L)/n.
2. Add noise drawn from Laplace(0, b) with scale b = Δf/ε.
3. Release f(D) + noise. The expected absolute error is b; the standard deviation (the typical spread around the average) √2·b.

Useful DP properties: **composition** (answering k queries at ε each costs up to kε in total — the privacy budget), and **post-processing** (anything computed from a DP output stays DP).

`code/privacy_dp.py`:

```
def laplace_count(true_count, eps, rng):
    return true_count + rng.laplace(0.0, 1.0 / eps)            # sensitivity of a count = 1

def laplace_mean(x, lo, hi, eps, rng):
    x = np.clip(x, lo, hi)                                      # clipping bounds the sensitivity
    sens = (hi - lo) / len(x)
    return x.mean() + rng.laplace(0.0, sens / eps)

gen = df.assign(zip=df.zip.str[:3] + "**",
                age=pd.cut(df.age, [0, 35, 120], labels=["<=35", ">35"]).astype(str))
```

Output:

```
raw table: k = 1
generalised (zip prefix + age band): k = 4
zip    age   sex
021**  <=35  F                    [flu]
       >35   M      [cancer, flu, heart]

exact: count(all)=54  count(all except Bob)=53  -> Bob's status leaked: 1
DP (eps=0.5 each): 52.8 vs 48.8 -> difference +4.0 (noise sd ~ 2.8 per query)

true count(hiv) = 54,  true mean income = 61.69k
eps     noisy count (3 draws)        mean |err| count   noisy mean income
0.01       96.8,   122.2,  -155.1         96.54          62.01k
0.1        58.2,    57.6,    60.9          9.83          62.20k
1.0        54.4,    54.3,    54.6          1.06          61.56k
10.0       53.9,    54.0,    53.9          0.10          61.68k
```

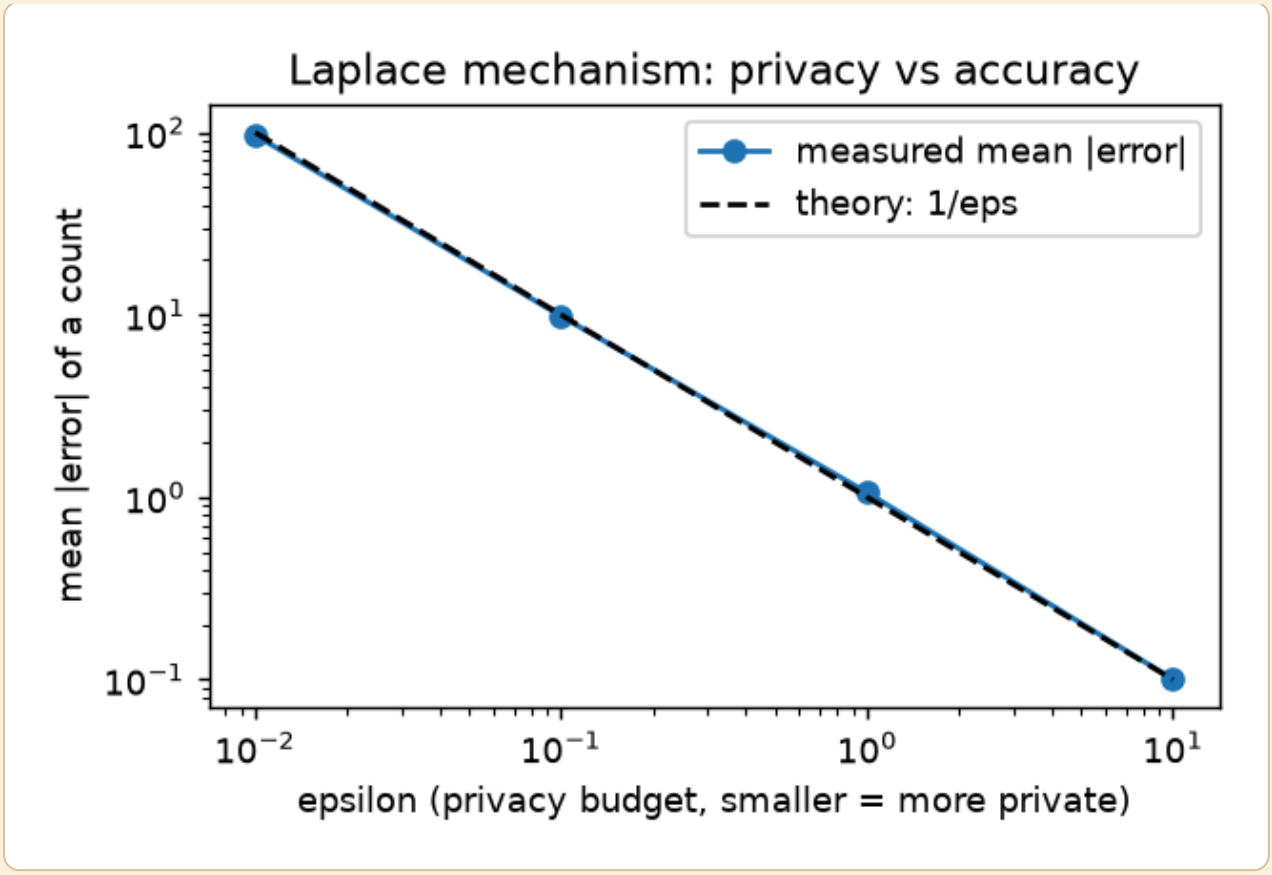


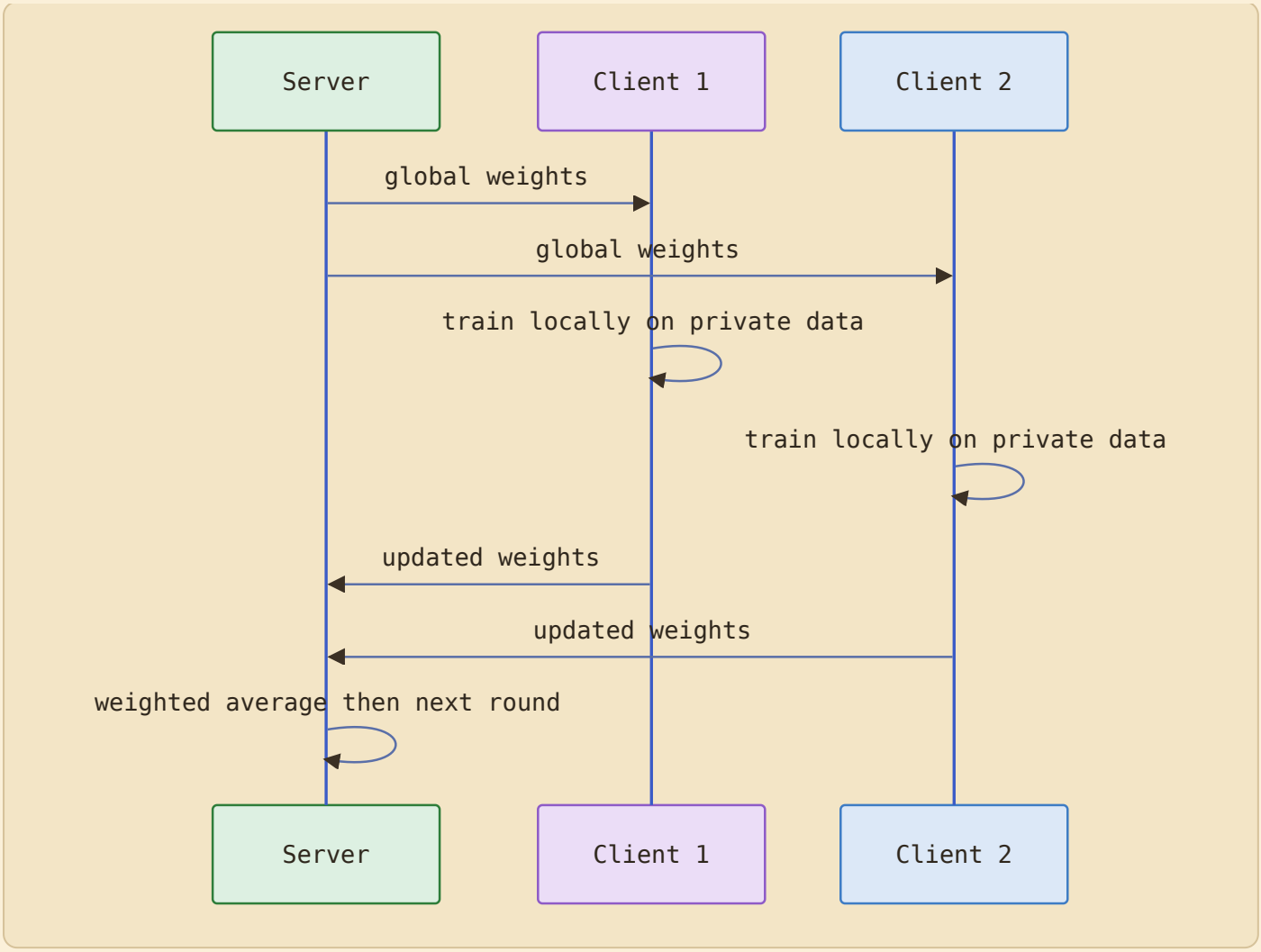

*Figure — the Laplace mechanism's mean absolute error on a count follows 1/ε: ten times more privacy costs ten times more error.*

Reading it:

1. Generalisation lifts k from 1 to 4, but everyone in the "021**, ≤35, F" group has flu, so knowing a woman aged ≤35 from that area is in the table reveals her diagnosis: a **homogeneity attack**, which l-diversity guards against.
2. Two harmless-looking exact counts ("all patients", "all except Bob") reveal Bob's HIV status: a **differencing attack**. With DP noise the difference (+4.0) says nothing reliable.
3. The mean-income query stays accurate even at ε = 0.01 because its sensitivity is tiny ((300 − 0)/10,000 = 0.03). **DP is cheap for large aggregates and expensive for small groups** — the central trade-off.

**DP in model training. DP-SGD** (Abadi et al., 2016) clips each example's gradient (the direction in weight space that increases the loss fastest) and adds Gaussian (bell-curve-shaped) noise before each update; the PyTorch library Opacus and TensorFlow Privacy implement it. It costs accuracy, especially on small datasets and rare classes.

**Federated learning (FL)** (McMahan et al., 2017) keeps raw data on devices or at institutions and moves the *model* instead: the server sends the global model to clients, each trains a few epochs (full passes over its data) locally and returns only its updated weights, and the server averages them weighted by client data size (**FedAvg**) before the next round.




`code/federated_avg.py` (three "hospitals" with different sizes and non-identical data):

```
for rnd in range(1, 31):
    updates, sizes = [], []
    for Xc, yc in clients:                                  # runs ON each client
        updates.append(local_train(w_global.copy(), Xc, yc))
        sizes.append(len(yc))
    w_global = np.average(updates, axis=0, weights=sizes)  # server: weighted average
```

Output:

```
round  1: global test accuracy = 0.786
round  5: global test accuracy = 0.815
round 10: global test accuracy = 0.823
round 30: global test accuracy = 0.826

centralised (all data pooled, 150 epochs): 0.824
client 0 alone (n=2500):            0.810
client 1 alone (n=1500):            0.767
client 2 alone (n=1000):            0.775
```

The federated model matches the pooled model (0.826 vs 0.824) and beats every client training alone — without any raw row leaving a client. Caveat: updates themselves can leak information (gradient inversion attacks, which rebuild training examples from shared updates), so production FL adds **secure aggregation** (the server sees only the sum of updates) and often DP noise.

## Trade-offs

| Technique | Protects against | Cost / limits |
|---|---|---|
| Data minimisation, access control, encryption | Breaches, misuse | Discipline; doesn't stop model leakage |
| Pseudonymisation | Casual identification | Easily reversed by linkage |
| k-anonymity (+ l-diversity, t-closeness) | Linkage on quasi-identifiers | Loses detail; weak against background knowledge; no formal guarantee |
| Differential privacy | Any attacker, any side information (formal bound) | Noise reduces accuracy; budget accounting; hard for small groups |
| Federated learning | Raw data centralisation | Communication cost, non-IID data (clients' data differ in distribution), updates can leak — combine with secure aggregation/DP |
| Synthetic data | Sharing raw records | Can memorise real records unless generated with DP |

## When not to use it

- Don't use DP on tiny datasets where the noise swamps the signal — restrict access instead.
- Don't claim "anonymous" because names were removed.
- Don't adopt FL when the data can legally and safely be centralised — it adds real engineering complexity.

## Common mistakes

- Forgetting composition: many queries at ε = 1 each add up to a weak guarantee.
- Not clipping inputs, so sensitivity is unbounded and the "DP" is fake.
- Logging raw PII in feature stores (databases that serve model inputs), debug logs or prompts (the text sent to a language model).
- Keeping training data forever "just in case" (violates storage limitation under GDPR).

## Real-world examples

The US Census Bureau protected 2020 census results with differential privacy (its "TopDown" algorithm). Apple uses local DP (noise is added on the user's device before anything is sent) to learn popular emoji and new words from devices; Google's RAPPOR (2014) collected Chrome statistics with local DP. Google's Gboard keyboard trains next-word prediction with federated learning. Hospitals use FL consortia for medical imaging models.

## In the interview

> **30-second answer:** "Removing names isn't anonymisation — quasi-identifiers link back to people. k-anonymity resists linkage without a formal guarantee. Differential privacy bounds any one person's influence: Laplace noise of scale sensitivity/ε, with ε budgeted across queries; DP-SGD applies it to training. Federated learning keeps data local and averages updates, with secure aggregation and DP because updates can leak."

- *Sensitivity of a mean of n values in [0, 100]?* → 100/n.

## Self-check

1. Count query, ε = 0.5. Laplace scale? → b = 1/0.5 = 2.
2. Ten queries at ε = 0.2 each. Total budget (basic composition)? → 2.0.

3. A 5-anonymous table where every record in a group has the same diagnosis — what's wrong? → No l-diversity: a homogeneity attack reveals the diagnosis.
4. What does the FedAvg server compute? → A data-size-weighted average of client model weights.

### TL;DR

Pseudonymised ≠ anonymous. k-anonymity resists linkage but not background knowledge; differential privacy adds noise of scale Δf/ε for a provable bound; federated learning moves the model, not the data.

---

## 19.7 Security: attacks on ML models

### What it is

**Adversarial ML** studies attackers who target the model itself rather than the servers around it. A **threat model** states what the attacker wants and can do: **white-box** (knows weights — the learned numbers inside the model — and code) or **black-box** (can only send queries and read answers); attacking at **training time** (the data) or **inference time** (the inputs, when the trained model is being used).

| Attack | When | Goal |
|---|---|---|
| **Adversarial examples (evasion)** | Inference | Make the model misclassify a crafted input (white-box: uses gradients; black-box: queries or a substitute model) |
| **Data poisoning** | Training | Degrade the model or plant a **backdoor** trigger |
| **Model extraction (stealing)** | Inference, via queries | Copy a paid or proprietary model |
| **Model inversion** | Inference, via queries | Reconstruct sensitive features or typical training inputs |
| **Membership inference** | Inference, via queries | Learn whether a record was in the training set |
| **Training-data extraction** | Inference, via queries | Make a generative model (one that produces text or images) emit memorised training text or images |

#### Why it matters

Models guard money and safety (fraud filters, malware detection, moderation, driving perception), so each attack maps to a real harm: fraud slipping through, a stolen model, or proof that a named person was in a cancer study.

### How it works

**1. Adversarial examples and FGSM.** Szegedy et al. (2013) found that tiny, targeted perturbations flip neural-network predictions. Goodfellow, Shlens & Szegedy (2015) explained it by the models' near-linear behaviour in high dimensions and introduced the **Fast Gradient Sign Method**:

$$x_{adv} = \text{clip}( x + \varepsilon \cdot \text{sign}( \nabla_x L(\theta, x, y) ), 0, 1 )$$

1. Compute the gradient of the loss with respect to the *input* pixels (not the weights).
2. Take its sign: +1 or −1 per pixel — the direction that increases the loss.
3. Move every pixel by ε in that direction (an $L_\infty$ budget of ε per pixel — a cap on the largest change to any single pixel) and clip to the valid range.

Many small nudges, each pointing the "wrong" way, add up to a large change in the logit (the raw score before it is turned into a probability). Their paper's famous image: a panda classified with 57.7% confidence becomes a "gibbon" at 99.3% after an imperceptible perturbation. **PGD** (Madry et al., 2018) repeats FGSM-style steps and is the standard strong attack. The main defence is **adversarial training**: include adversarial examples in each training batch.

`code/fgsm_attack.py` (an MLP — multi-layer perceptron, a basic fully connected neural network — on sklearn's 8×8 digits, pixels scaled to [0, 1]):

```
def fgsm(model, x, y, eps):
    x = x.clone().requires_grad_(True)
    loss = nn.functional.cross_entropy(model(x), y)
    grad, = torch.autograd.grad(loss, x)
    return (x + eps * grad.sign()).clamp(0, 1).detach()

# adversarial training: each batch = clean examples + their FGSM versions
if adversarial:
    xb = torch.cat([xb, fgsm(model, xb, yb, eps)]); yb = torch.cat([yb, yb])
```

Output:

```
eps    plain-model acc   adversarially-trained acc
0.0    0.976             0.989
0.05   0.796             0.946
0.1    0.365             0.826
0.2    0.061             0.298
0.3    0.017             0.019

example: true=1  clean pred=1 (1.00)  adversarial pred=8 (1.00)
```

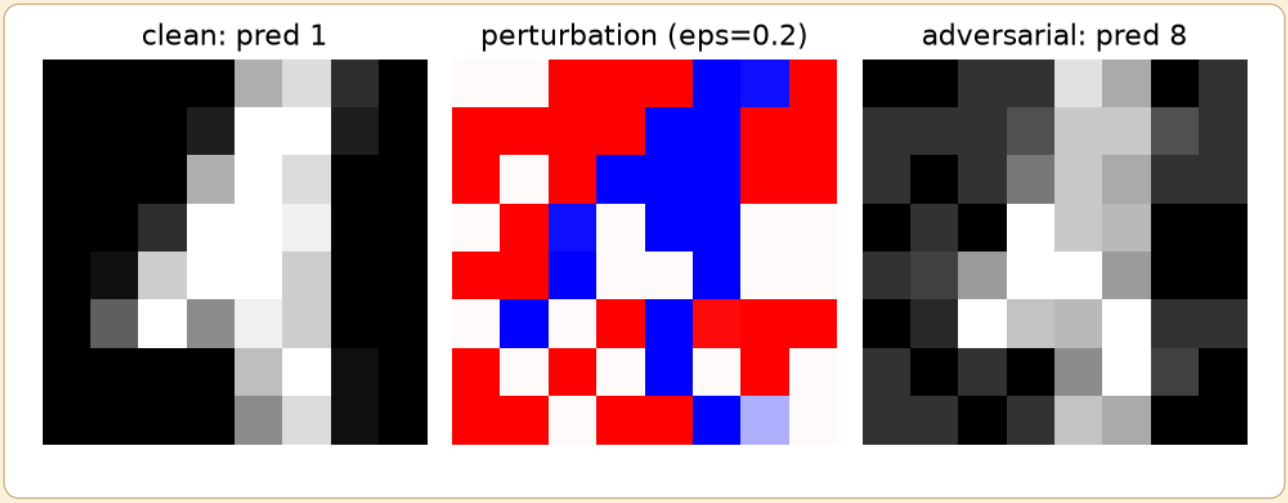



*Figure — a "1" (left), the FGSM perturbation with ε = 0.2 (middle, red = brighter, blue = darker), and the result classified as "8" with full confidence (right).*

Reading it: a per-pixel budget of 0.1 cuts the plain model from 97.6% to 36.5%. Adversarial training at ε = 0.1 keeps 82.6% — but gives little protection at larger budgets it never trained on. (On 8×8 images an ε of 0.2 is visible; on high-resolution photos far smaller budgets suffice and the change is invisible.)

**Physical-world versions exist**: Eykholt et al. (2018) made stop signs misread by adding sticker patterns; adversarial patches and glasses frames fool face recognition.

**2. Data poisoning.** The attacker corrupts training data. **Label flipping** changes labels; a **backdoor** (e.g. BadNets, Gu et al., 2017) adds a trigger pattern to some examples with a chosen label, so the model behaves normally until it sees the trigger. Risk grows when training on scraped web data, user

feedback or crowd-sourced labels. Microsoft's Tay chatbot (2016) was withdrawn within a day after users deliberately taught it offensive content — a well-known case of manipulated learning from user input.

**3. Model extraction.** Tramèr et al. (2016) showed that prediction APIs (interfaces other programs call over the network — see System Design Fundamentals) can be copied by querying them and training a surrogate on the answers. The stolen copy also enables white-box attacks against the original.

**4. Model inversion.** Fredrikson et al. (2015) reconstructed recognisable face images of individuals from a face-recognition model using its confidence scores.

**5. Membership inference.** Shokri et al. (2017): models are more confident on examples they trained on, so high confidence on the true label suggests membership. Overfitting is the main driver. Carlini et al. (2021) extracted verbatim memorised training text — including personal contact details — from GPT-2 (OpenAI's 2019 language model).

`code/attacks_demo.py`:

```
# poisoning: random vs targeted (class 1 relabelled as 0)
y_tgt[rng.choice(ones, int(frac * len(ones)), replace=False)] = 0
# extraction: query the victim, train a copy on its answers
answers = victim.predict(Q); stolen = DecisionTreeClassifier(max_depth=10).fit(Q, answers)
# membership inference: confidence on the true label, members vs non-members
conf_in = model.predict_proba(X_in)[np.arange(len(y_in)), y_in]
auc = roc_auc_score(np.r_[np.ones(len(conf_in)), np.zeros(len(conf_out))], np.r_[conf_in, conf_out])
```

Output:

```
1) label-flip poisoning (logistic regression, test accuracy / recall of class 1)
   poisoned  0%:  random flips 0.843/0.838   targeted 1->0 0.843/0.838
   poisoned 10%:  random flips 0.846/0.835   targeted 1->0 0.841/0.778
   poisoned 20%:  random flips 0.843/0.854   targeted 1->0 0.812/0.689
   poisoned 30%:  random flips 0.840/0.835   targeted 1->0 0.769/0.574

2) model extraction - victim test accuracy 0.870
     100 queries: agreement with victim - random queries 0.659, look-alike queries 0.820
    1000 queries: agreement with victim - random queries 0.720, look-alike queries 0.892
    5000 queries: agreement with victim - random queries 0.845, look-alike queries 0.933

3) membership inference (attack = 'high confidence on the true label means member')
   overfit forest (no depth limit)  train acc 1.000  test acc 0.902  attack AUC 0.842
   regularised forest (depth 4)     train acc 0.896  test acc 0.851  attack AUC 0.556
```

Reading it: **random** label noise barely hurts, but **targeted** flipping of 10–30% of positives drags recall from 0.84 to 0.57, so a fraud filter can be taught to miss exactly the attacker's class. **Extraction** reaches 93% agreement with 5,000 realistic queries. **Membership inference** is strong (AUC 0.84) against an overfit model and near chance (0.56) after regularisation (constraints that keep a model simple, here a depth limit): generalisation is also a privacy defence.

**6. Generative-AI misuse. Deepfakes** (fake video, audio or images impersonating real people for fraud, blackmail or non-consensual imagery) and cheap **misinformation at scale** are harms of a different kind. Responses combine detection classifiers (an arms race), **provenance** (a verifiable record of where content came from) standards such as C2PA "Content Credentials", watermarking (hidden marks embedded in generated content) (e.g. Google DeepMind's SynthID), platform policies, labelling duties such as the EU AI Act's (§19.9), and media literacy.

### Defences at a glance

| Threat | Defences |
|---|---|
| Evasion | Adversarial training (PGD), input preprocessing (weak alone), certified robustness (randomised smoothing), ensembles, human review for high-stakes cases |
| Poisoning / backdoors | Data provenance and access control, outlier and label-noise detection, robust training, dataset versioning, backdoor scanning |
| Extraction | Rate limits, authentication, returning labels instead of full probabilities, query-pattern monitoring, watermarking the model |
| Inversion / membership | Regularisation, DP-SGD, limiting confidence outputs, deduplicating training data |
| Generative misuse | Provenance and watermarking, usage policies, abuse monitoring, red-teaming (testers deliberately trying to break the system) |

## Common mistakes

- Evaluating robustness only against weak attacks (e.g. FGSM) — test with PGD and adaptive attacks.
- Relying on "gradient masking" defences that only hide gradients; they are routinely broken.
- Returning full-precision probability vectors from public APIs when a label would do.
- Training on unvetted scraped or user-feedback data with no provenance.

## Real-world examples

Spam and malware detection are adversarial by nature (attackers constantly probe filters). MITRE ATLAS catalogues real attacks on ML systems; the OWASP (an open web-security community) Top 10 for LLM (large language model) Applications lists data/model poisoning and sensitive-information disclosure among its risks.

## In the interview

> **30-second answer:** "Evasion uses adversarial examples — FGSM moves each input feature by ε along the sign of the loss gradient — and is countered by adversarial training. Poisoning corrupts training data or plants backdoors; defend with provenance and anomaly checks. Through a query API attackers can extract, invert or probe the model for membership; rate limits, label-only outputs, regularisation and DP help."

- *Why do adversarial examples exist?* → High-dimensional near-linear models: many tiny aligned changes add up to a large logit change.
- *Cost of adversarial training?* → Slower training (extra attack steps), sometimes lower clean accuracy, and robustness only within the trained budget.

## Self-check

1. FGSM uses the gradient with respect to what? → The input, not the weights.
2. Which is more damaging: random 20% label noise or targeted 20% flipping of one class? → Targeted (recall fell to 0.69 vs ~0.85).
3. Two cheap defences against model extraction? → Rate limiting / authentication and returning only top labels.

## TL;DR

Adversarial training against evasion; provenance against poisoning; rate limits and label-only outputs against extraction; regularisation and DP against membership inference; provenance and policy against generative misuse.

## 19.8 Documentation: model cards and datasheets

### What it is

- A **model card** (Mitchell et al., 2019) is a short document shipped with a model describing what it is for, how it was evaluated — *broken down by group* — and its limits.
- A **datasheet for datasets** (Gebru et al., 2018; published in *Communications of the ACM* in 2021) documents how a dataset was created, what's in it and how it should (not) be used — modelled on electronics datasheets.

### Why it matters

Most harms come from a model being used outside the conditions it was built for. Written, versioned documentation makes intended use, known gaps and evaluation results visible to downstream teams, auditors and regulators (the EU AI Act requires technical documentation for high-risk systems).

### How it works

**Model card sections** (following Mitchell et al.):

| Section | Our credit model |
|---|---|
| Model details, intended use | Logistic regression v1.3, owner, date; pre-screening loan applications with human review, *not* employment or insurance |
| Factors, metrics | Groups: sex, age band, region and intersections; AUC, per-group TPR/FPR, disparate impact, calibration |
| Training and evaluation data | Sources and time range, known label issues; held-out 2024 applications (n = 6,000) |
| Quantitative analyses | Per-group metrics with confidence intervals |
| Ethics, caveats | Proxy features (zip region), mitigation applied, residual gaps; re-audit quarterly, not for other populations |

**Datasheet questions** are grouped as *motivation* (why, by whom, funded by whom), *composition* (contents, gaps, sensitive data, identifiability), *collection process* (how, consent), *preprocessing and labelling*, *uses*, *distribution* and *maintenance*.

### Common mistakes

- Writing it once for launch and never updating it — keep it in the repo (the project's version-controlled code repository), generate the metric tables in the training pipeline, and version it with each release.
- Only aggregate metrics — the whole point is disaggregated (per-group) results.
- Omitting "out-of-scope uses".

### Real-world examples

Hugging Face (a popular site for sharing models) model cards (a README on every model page), Google's Model Card Toolkit, and system cards published by AI labs for large language models; Hugging Face also supports dataset cards.

### In the interview

> **30-second answer:** "A model card documents intended use, out-of-scope uses, training and evaluation data, per-group metrics and limitations; a datasheet documents a dataset's

motivation, composition, collection, labelling, uses and maintenance. I generate the metric sections automatically in the pipeline and version them with the model."

### Self-check

1. What distinguishes a model card's metrics from an ordinary eval report? → They are disaggregated by group and conditions.
2. Name three datasheet sections. → Motivation, composition, collection process (also preprocessing, uses, distribution, maintenance).

### TL;DR

Model cards and datasheets make intended use, data provenance and per-group performance explicit and versioned.

---

## 19.9 Regulation overview

*This is an engineer's orientation, not legal advice. Laws change — check the official texts and your legal team.*

### What it is and why it matters

Laws govern both the **data** ML uses (privacy law) and the **systems** it produces (AI and anti-discrimination law); engineering artefacts — logs, human review, documentation, explanations — are often what compliance actually requires.

**GDPR (EU General Data Protection Regulation, applies since 25 May 2018)**

- Covers **personal data** of people in the EU, including processing by non-EU companies serving them. Truly anonymous data is out of scope; pseudonymised data is not.
- **Principles** include purpose limitation, **data minimisation**, accuracy, storage limitation and accountability; every use needs a **lawful basis** (consent, contract, legitimate interests…), with stricter rules for "special categories" such as health, biometrics and ethnicity.
- **Rights**: access, rectification, **erasure** ("right to be forgotten"), restriction, portability, objection.
- **Article 22**: people have the right not to be subject to decisions based *solely* on automated processing with legal or similarly significant effects; the exceptions (contract, law, explicit consent) require safeguards such as human intervention and the chance to give their view and contest. Articles 13–15 require "meaningful information about the logic involved"; whether that is a full "right to explanation" is debated.
- High-risk processing needs a **DPIA** (data protection impact assessment); fines reach €20 million or 4% of worldwide annual turnover, whichever is higher.

**EU AI Act (Regulation (EU) 2024/1689)**

Entered into force on 1 August 2024, with obligations phased in. It classifies AI systems by **risk**:

| Tier | Examples | Obligations |
|---|---|---|
| **Unacceptable (prohibited)** | Social scoring; harmful manipulation or exploitation of vulnerabilities; untargeted scraping of face images; emotion recognition at work or school; biometric inference of sensitive traits; crime prediction purely from profiling; real-time remote biometric identification in public by police (narrow exceptions) | Banned (applied from 2 February 2025) |
| **High risk** | Safety components of regulated products (Annex I), and Annex III areas: biometrics, critical infrastructure, education, **employment**, access to essential services including **credit scoring**, law enforcement, migration, justice | Risk management, data governance (including bias checks), documentation, logging, **human oversight**, accuracy and robustness, conformity assessment, registration |
| **Transparency (limited) risk** | Chatbots, AI-generated or manipulated content (deepfakes), emotion recognition | Tell people they're interacting with AI; label synthetic content |
| **Minimal risk** | Spam filters, game AI, most business ML | No new obligations (voluntary codes) |

Separately, providers of **general-purpose AI models** (e.g. large language models) have documentation and copyright-policy duties from 2 August 2025, with extra duties for models posing systemic risk. Top fines reach €35 million or 7% of worldwide turnover for prohibited practices.

**Timeline update (2026).** The EU's "Digital Omnibus on AI" (Regulation (EU) 2026/1744, in force since 27 July 2026) postponed the high-risk obligations: to **2 December 2027** for stand-alone Annex III systems and **2 August 2028** for AI in Annex I products. It also added prohibitions on AI systems that generate non-consensual intimate imagery or child sexual abuse material (applying from 2 December 2026), and gave generative systems placed on the market before 2 August 2026 until 2 December 2026 to meet the Article 50 duty to mark their output as AI-generated. Check the consolidated text for current dates.

### Other rules worth knowing

- **CCPA/CPRA (California)**: rights to know, delete, correct and opt out of the sale or sharing of personal information.
- **US anti-discrimination law**: disparate *treatment* (using a protected attribute) and disparate *impact* (a neutral rule with unequal effect and no business necessity); under ECOA/Regulation B lenders must give specific denial reasons, and the CFPB (the US Consumer Financial Protection Bureau) says complex algorithms are no excuse (Circular 2022-03).
- **NYC Local Law 144** (enforced from July 2023): automated employment decision tools used for NYC jobs need an annual independent bias audit, published results, and candidate notice.
- **Sector rules** such as HIPAA (US health data) and model-risk-management guidance for banks (US SR 11-7).
- **Frameworks** (voluntary): NIST (the US national standards agency) AI Risk Management Framework (2023), ISO/IEC 42001 (AI management systems, 2023).

## In the interview

**30-second answer:** "GDPR governs personal data: lawful basis, minimisation, rights such as erasure, and Article 22 safeguards for solely automated significant decisions. The EU AI Act is risk-based: some uses are banned; high-risk systems such as credit scoring and hiring need risk management, data governance, documentation, logging and human oversight; chatbots and deepfakes carry transparency duties. In the US, fair-lending and employment law (adverse-action reasons, disparate impact, NYC's bias audits) shape design."

### Self-check

1. Is a hiring-screening model high risk under the EU AI Act? → Yes, employment is an Annex III area.
2. Is pseudonymised data covered by GDPR? → Yes.
3. What does Article 22 give people? → Protection from solely automated significant decisions, with rights to human intervention and to contest.

### TL;DR

GDPR governs personal data (lawful basis, minimisation, rights, Art. 22); the EU AI Act sorts systems into risk tiers, with credit and hiring high-risk; US fair-lending and employment law add their own duties.

---

## 19.10 Real-world incidents

**COMPAS recidivism scores (2016)**

- **What happened**: ProPublica analysed COMPAS recidivism-risk scores (predictions of whether someone will reoffend) (by Northpointe, now Equivant) for defendants in Broward County, Florida, checking who was re-arrested within two years. Among defendants who did **not** reoffend, Black defendants were labelled higher risk about twice as often as white defendants (44.9% vs 23.5%); among those who **did** reoffend, white defendants were more often labelled lower risk (47.7% vs 28.0%).
- **The response**: Northpointe argued the scores were equally predictive for both groups — a given score corresponded to similar reoffence rates (calibration/predictive parity).
- **Lesson**: *both sides were right about their own metric*. With different base rates, calibration and equal error rates cannot both hold — this debate triggered the impossibility proofs (§19.2). Also: "re-arrest" is a proxy label shaped by policing (measurement bias).

**Amazon's experimental résumé-screening tool (reported 2018)**

- **What happened**: Reuters reported that from 2014 Amazon built a tool to rate candidates, trained on ten years of résumés, mostly from men. It learned to penalise the word "women's" (as in "women's chess club captain") and downgraded graduates of two all-women's colleges. Engineers removed those terms but could not rule out other proxies, and the team was disbanded; Amazon said recruiters never used the tool to evaluate candidates.
- **Lesson**: historical bias in labels, and proxies — removing explicit terms doesn't remove the pattern.

**Apple Card credit limits (2019–2021)**

- **What happened**: In November 2019 entrepreneur David Heinemeier Hansson complained publicly that his credit limit was 20 times his wife's despite shared finances; Steve Wozniak reported a similar experience. New York's Department of Financial Services (NYDFS) investigated Goldman Sachs, the issuer.
- **Outcome**: In March 2021 the NYDFS found **no violation of fair-lending law** (individual credit histories explained the differences) but criticised the lack of transparency and poor customer service.
- **Lesson**: even a lawful model loses trust if it cannot explain its decisions; spouses with shared finances can still have very different individual credit files.

Gender Shades and the Obermeyer health-risk algorithm (§19.1) complete the set of cases interviewers most often cite.

### Self-check

1. Which fairness criterion did ProPublica test, and which did Northpointe defend? → Equal error rates (FPR/FNR) vs calibration/predictive parity.
2. What did the NYDFS conclude about Apple Card? → No fair-lending violation, but transparency and service shortcomings.

### TL;DR

COMPAS = the impossibility result in real life; Amazon = historical bias and proxies; Apple Card = explainability and trust.

---

## 19.11 Responsible-ML checklist

Use this before launch and at every major retrain.

**Problem framing**

- [ ] Is ML appropriate? Who could be harmed? High-risk under the EU AI Act or similar law?
- [ ] Does the label measure what we care about? Which fairness criterion fits the harm (agreed with legal/domain experts)?

**Data**

- [ ] Datasheet: source, lawful basis, collection, labelling, known gaps; coverage audit per group and intersection.
- [ ] PII minimised and access-controlled; sensitive attributes stored separately for auditing; provenance and poisoning checks.

**Model and evaluation**

- [ ] Per-group metrics with confidence intervals; proxy check; mitigation chosen, its cost measured, all criteria re-audited.
- [ ] Explanations reviewed for leakage and proxies; reason codes where decisions affect people.
- [ ] Robustness tests where attackers exist; membership-inference risk; DP/FL where needed.

**Deployment and monitoring**

- [ ] Model card with intended and out-of-scope uses; human oversight and an appeal path.
- [ ] API hardening (authentication — checking who the caller is; rate limits — capping requests per caller; minimal outputs).
- [ ] Per-group monitoring, drift and feedback-loop checks; incident and rollback (return to the previous model) plan; scheduled re-audits.

---

## 19.12 Interview Q&A

The sections above each end with their own interview answers; these are the cross-cutting questions that combine them.

1. **Your model is 94% accurate. Is it fair?** → Unknown until you slice by group: selection rate, TPR, FPR, PPV and calibration per group, the gaps, and a proxy check. Accuracy alone hides group disparities.
2. **Explain SHAP to a product manager.** → It splits one prediction into per-feature contributions that add up from the average prediction to this one: "income added +0.18, a prior default took away

−0.09".

3. **Is hashing user IDs anonymisation?** → No, it is pseudonymisation; linkage attacks re-identify people and GDPR still applies.
4. **Someone is scraping our pricing model's API. Risks and defences?** → Model extraction, which then enables white-box attacks. Rate limits, authentication, label-only responses, query-pattern monitoring, contractual terms.
5. **Design question: build a fair hiring screener.** → Question whether to automate at all; define the label carefully (job performance, not past hiring decisions); audit data coverage; choose a criterion (often selection-rate parity plus equal TPR) with legal input; mitigate and re-audit; give candidates reasons; keep human review; run bias audits (NYC LL 144 if applicable) and meet EU AI Act high-risk duties; monitor per-group outcomes after launch.

---

## Chapter TL;DR

- **Bias** enters through history, sampling, measurement, labels, aggregation, evaluation, deployment and feedback loops; a biased label can't be fixed by a better model.
- **Fairness criteria** (parity, equal opportunity, equalized odds, predictive parity, calibration) conflict when base rates differ — choose by harm, audit per group, test for proxies.
- **Mitigate** before (reweighing, better data), during (penalties, constraints) or after training (group thresholds); measure the accuracy cost and re-audit.
- **Explain** globally (permutation importance, PDP/ICE) and locally (SHAP, LIME, counterfactuals) — never as causal claims.
- **Privacy**: pseudonymised ≠ anonymous; k-anonymity + l-diversity; differential privacy (noise $\Delta f/\varepsilon$, budgets compose), DP-SGD, federated averaging with secure aggregation.
- **Security**: evasion (FGSM/PGD, adversarial training), poisoning, extraction, inversion, membership inference, generative misuse.
- **Document and comply**: model cards, datasheets, GDPR Art. 22, EU AI Act risk tiers, US fair-lending and hiring-audit rules. Remember COMPAS, Amazon and Apple Card.